

Repository: <https://github.com/BhavyaJaiswal97/crispr-grna-visualizer> | Design assets: [/design/](#) (Draw.io sources, PNG exports, UI screens)

1. Introduction and Scope

The CRISPR gRNA Visualizer is a web application that ingests a target DNA sequence, locates Protospacer Adjacent Motif (PAM) sites, extracts 20-bp spacer candidates, scores each candidate for on-target efficiency and off-target specificity, and renders the results on an interactive genomic track map. This document takes the 25 user stories and six wireframes produced in DA-1 and turns them into a concrete design: how the system is decomposed into modules, which abstractions hold the decomposition together, how the modules communicate, and how the user interface expresses the workflow. The emphasis throughout is on *why* a structure was chosen and how it absorbs the changes the project is expected to face — new Cas nucleases, new scoring models, and a move from in-sequence search to whole-genome alignment.

Section	Question it answers
2. Design principles	How abstraction, modularity, cohesion and low coupling were applied, with code.
3. High-level architecture	Which architectural style was chosen, why, and what the runtime flow looks like.
4. User interface design	The six screens and the usability reasoning behind them.
5. Design decisions	Five key choices, the alternatives rejected, and the trade-offs accepted.
6. Maintainability	How the design behaves when the requirements change.

2. Design Principles Applied

2.1 Abstraction — program to a concept, not to a nuclease

The single most volatile requirement in this project is the CRISPR system itself. SpCas9 reads a 3'-side `NGG` PAM with a 20-bp spacer; Cas12a reads a 5'-side `TTTV` PAM with a 23-bp spacer. If PAM logic were written inline in the analysis endpoint, every new nuclease would mean editing that endpoint. Instead the design names the *concept* — a nuclease is anything that can find PAM sites in a sequence and extract a spacer next to one — and expresses it as an abstract base class. Everything above it (the service, the API, the UI) deals only with that contract; the biology lives inside the subclasses.

```
# backend/core/nuclease.py
from abc import ABC, abstractmethod
from dataclasses import dataclass

IUPAC = {"N": "[ACGT]", "V": "[ACG]", "R": "[AG]", "Y": "[CT]"}

@dataclass(frozen=True)
class PAMSite:
    start: int          # 0-based index on the analysed strand
    strand: str          # '+' or '-'
    pam: str

class Nuclease(ABC):
    """Contract every CRISPR system must satisfy. No I/O, no framework imports."""

    name: str
    spacer_len: int
    pam_motif: str      # IUPAC, e.g. 'NGG'
    pam_is_3prime: bool

    @property
    def pam_regex(self) -> str:
        return "".join(IUPAC.get(base, base) for base in self.pam_motif)

    @abstractmethod
    def find_pam_sites(self, sequence: str) -> list[PAMSite]: ...

    @abstractmethod
    def extract_spacer(self, sequence: str, site: PAMSite) -> str | None: ...

class SpCas9(Nuclease):
    name, spacer_len, pam_motif, pam_is_3prime = "SpCas9", 20, "NGG", True
    # find_pam_sites / extract_spacer implemented once for all 3'-PAM enzymes

class Cas12a(SpCas9):
    # 5'-PAM variant: only the two fields change
    name, spacer_len, pam_motif, pam_is_3prime = "Cas12a", 23, "TTTV", False
```

The same idea is applied twice more. **Scoring** is abstracted behind `ScoringStrategy`, so the Hsu/MIT specificity model and the Doench on-target efficiency model are interchangeable plug-ins rather than `if` branches. **Persistence** is abstracted behind `CacheRepo` and `ProjectRepo`, so the domain never imports `redis` or `sqlalchemy`. Abstraction here is not decoration — it is what makes user story 9 ("optimised, swappable alignment engine") and the "Could Have" Cas12a item implementable without touching tested code.

```
# backend/core/scoring.py
class ScoringStrategy(ABC):
    name: str
    @abstractmethod
    def score(self, guide: GuideRNA, context: ScoringContext) -> float: ...

class MITSpecificityScorer(ScoringStrategy):
    """Hsu et al. position-weighted mismatch penalty; seed region weighted hardest."""
    name = "mit_specificity"
    W = [0.000, 0.000, 0.014, 0.000, 0.000, 0.395, 0.317, 0.000, 0.389, 0.079,
          0.445, 0.508, 0.613, 0.851, 0.732, 0.828, 0.615, 0.804, 0.685, 0.583]

    def score(self, guide, context):
        total = 100.0
        for hit in context.off_targets:
            product = 1.0
            for pos in hit.mismatch_positions:          # 1-based, PAM-proximal = high
                product *= (1 - self.W[pos - 1])
            total -= product * 100 / max(len(context.off_targets), 1)
        return max(0.0, round(total, 1))
```

2.2 Modularity — one folder, one responsibility

The repository is decomposed so that a reader can predict where a change belongs before opening any file. Each package has a single reason to change, and dependencies point in one direction only: `api` → `services` → `core`, with `infrastructure` attached at the edge through interfaces. The domain core is a pure-Python island: it can be unit-tested with `pytest` in milliseconds, with no server, database or container running.

```
crispr-grna-visualizer/
+- backend/
| +- api/v1/           routers: analysis.py, projects.py, export.py, health.py  (HTTP only)
| +- schemas/         Pydantic request/response DTOs - the contract with the frontend
| +- services/         guide_design_service.py, export_service.py (orchestration only)
| +- core/             nuclease.py, scoring.py, guide.py, alignment.py (pure domain logic)
| +- infrastructure/   redis_cache.py, sql_project_repo.py, models.py (adapters)
| +- tests/            test_nuclease.py, test_scoring.py, test_api.py
+- frontend/src/
| +- components/       SequenceInput/ CandidateTable/ TrackMap/ OffTargetDrawer/
| +- hooks/            useAnalysis.js, useProjects.js      (data fetching + state)
| +- api/              client.js                          (single Axios instance)
| +- pages/            Dashboard.jsx, Results.jsx, Projects.jsx
+- design/             *.drawio sources, PNG exports, UI screens  (this document's assets)
+- docker-compose.yml, .github/workflows/ci.yml, README.md
```

Module	Responsibility	Changes when...
core/	PAM detection, spacer extraction, GC content, mismatch and efficiency scoring	the biology or the scoring model changes
services/	Sequencing of steps, caching policy, ranking, assembling the report	the workflow changes (e.g. add a filter stage)
api/ + schemas/	HTTP routing, validation, status codes, OpenAPI contract	the wire format or an endpoint changes
infrastructure/	Redis and PostgreSQL adapters implementing the repo interfaces	a storage technology is swapped
frontend/components/	Presentation only; props in, events out	the visual design changes
frontend/hooks/	Server state, loading and error handling	an endpoint or its shape changes

2.3 Cohesion — functionally cohesive units

Every module above sits at the *functional cohesion* level: its elements exist to compute one well-defined thing. `scoring.py` contains nothing but scoring; it does not fetch, log or serialise. `TrackMap` draws the canvas and emits a `onCandidateSelect` event; it does not know what an API is. The clearest test applied during design was the "and" test — if a module's description needs the word "and" to be honest, it was split. That is why export logic lives in `export_service.py` rather than inside `GuideDesignService`, and why the off-target drawer is a separate component from the candidate table even though DA-1 wireframe 4 shows them on the same screen.

```
// frontend/src/hooks/useAnalysis.js - all server state in one cohesive hook
export function useAnalysis() {
  const [state, setState] = useState({ status: "idle", report: null, error: null });

  const run = useCallback(async (payload) => {
    setState({ status: "loading", report: null, error: null });
    try {
      const report = await api.analyse(payload); // api/client.js owns the URL
      setState({ status: "done", report, error: null });
    } catch (e) {
      setState({ status: "error", report: null, error: toUserMessage(e) }); // story 25
    }
  }, []);

  return { ...state, run };
}

// components/CandidateTable/CandidateTable.jsx - presentational, zero fetching
export default function CandidateTable({ candidates, onSelect }) { ... }
```

2.4 Low coupling — dependency inversion at every seam

Coupling is controlled by three rules. First, **the service depends on interfaces, not implementations**: it receives a `Nuclease`, a list of `ScoringStrategy` and a `CacheRepo` through its constructor, so it never imports `Redis`. Second, **FastAPI's dependency injection supplies the concrete objects at the edge**, which means tests inject an in-memory fake cache and no container is needed. Third, **the frontend and backend are coupled only by JSON DTOs** defined in `schemas/` and published automatically as OpenAPI, so either side can be rewritten as long as the contract holds. The coupling that remains is data coupling (plain arguments and DTOs) — the loosest useful kind; there is no shared global state and no control-flag parameters.

```
# backend/services/guide_design_service.py
class GuideDesignService:
    def __init__(self, nuclease: Nuclease,
                  scorers: list[ScoringStrategy],
                  cache: CacheRepo):
        # interfaces only
        self._nuclease, self._scorers, self._cache = nuclease, scorers, cache

    def analyse(self, sequence: str, background: str | None) -> AnalysisReport:
        key = sha256(f"{self._nuclease.name}:{sequence}:{background}".encode()).hexdigest()
        if (cached := self._cache.get(key)):
            # cache-aside, story 24
            return AnalysisReport.parse_raw(cached)

        guides = [g for site in self._nuclease.find_pam_sites(sequence)
                  if (g := GuideRNA.build(sequence, site, self._nuclease))]
        for g in guides:
            ctx = ScoringContext(off_targets=find_off_targets(g, background))
            g.scores = {s.name: s.score(g, ctx) for s in self._scorers}

        report = AnalysisReport(candidates=rank(guides))
        self._cache.set(key, report.json(), ttl=3600)
        return report

# backend/api/v1/analysis.py - the only place that knows about concrete classes
@router.post("/analyze", response_model=AnalysisResponse, status_code=200)
def analyze(req: AnalysisRequest, svc: GuideDesignService = Depends(build_service)):
    return svc.analyse(req.sequence, req.background)

def build_service(nuclease: str = "SpCas9") -> GuideDesignService:
    return GuideDesignService(NucleaseFactory.create(nuclease),
                              [MITSpecificityScorer(), DoenchEfficiencyScorer()],
                              RedisCacheRepo(settings.redis_url))
```

Principle in one line: adding Cas12a means adding one subclass and one factory entry — no edit to the service, the router, the schemas or the UI. That is the Open/Closed Principle falling out of abstraction plus dependency inversion, and it is the practical reason the three principles above were worth the extra indirection.

3. High-Level Architecture

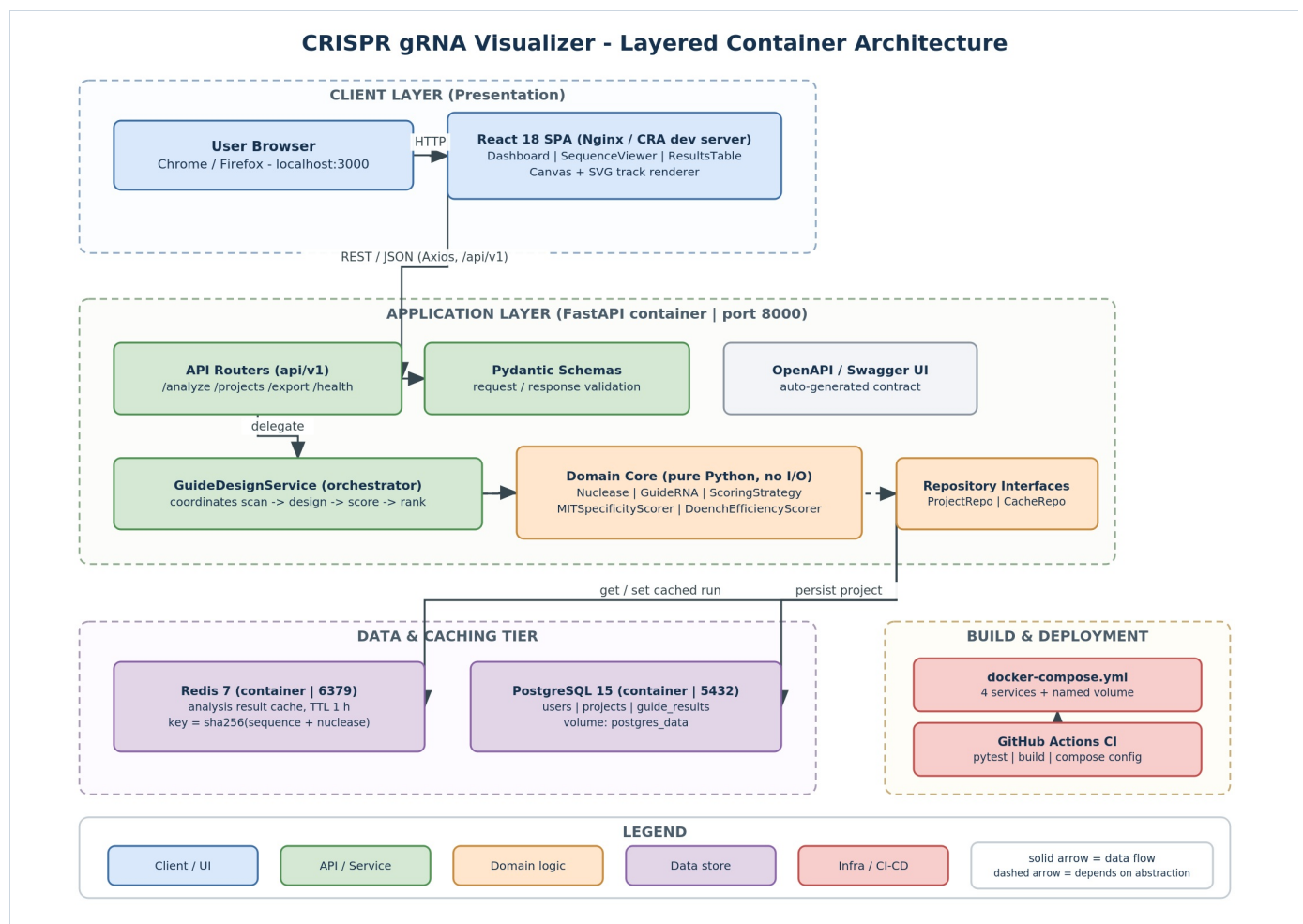


Figure 1 — Container and layer architecture (editable source: </design/crispr-architecture.drawio>, page 1).

3.1 Chosen style: a layered domain core inside a containerised client-server system

The system is a **three-tier client-server deployment** (React SPA → FastAPI service → Redis + PostgreSQL), and the server itself is organised in **layers** — API, application/service, domain, infrastructure — with dependencies pointing inward toward a framework-free domain core, as in the Ports-and-Adapters variant of layering. A **cache-aside** pattern sits between the service and Redis.

Why this style. The workload is a single, CPU-bound, stateless computation (scan, score, rank) that is requested by one user

at a time and is perfectly cacheable, and the algorithms — not the plumbing — are where the project's value and its future changes live. Layering keeps those algorithms isolated and unit-testable, lets the compute-heavy part be scaled or moved to a Celery worker later by swapping one adapter, and matches the team size (one developer) with the least operational overhead. Containerising the four processes gives story 20's one-command local setup without pushing the design into distributed territory.

Alternative considered	Why rejected
Single-file Flask/Streamlit monolith with logic in the route handlers	Fastest to demo, but PAM and scoring logic becomes untestable and un-swappable; the Cas12a and BLAST roadmap items would each force edits to working endpoints.
Microservices (separate scan, score, export services)	Adds network hops, retries and deployment surface for a workload that is one short synchronous call. Premature distribution for a single-developer project.
Server-rendered pages (Jinja templates)	The interactive track map, zoom/pan and hover tooltips (stories 11-14) need client-side state; full-page reloads would break the <500 ms interaction target.
Serverless functions	Cold starts and payload limits conflict with 50 kb sequence uploads and the <3 s latency metric; local Docker parity would be lost.

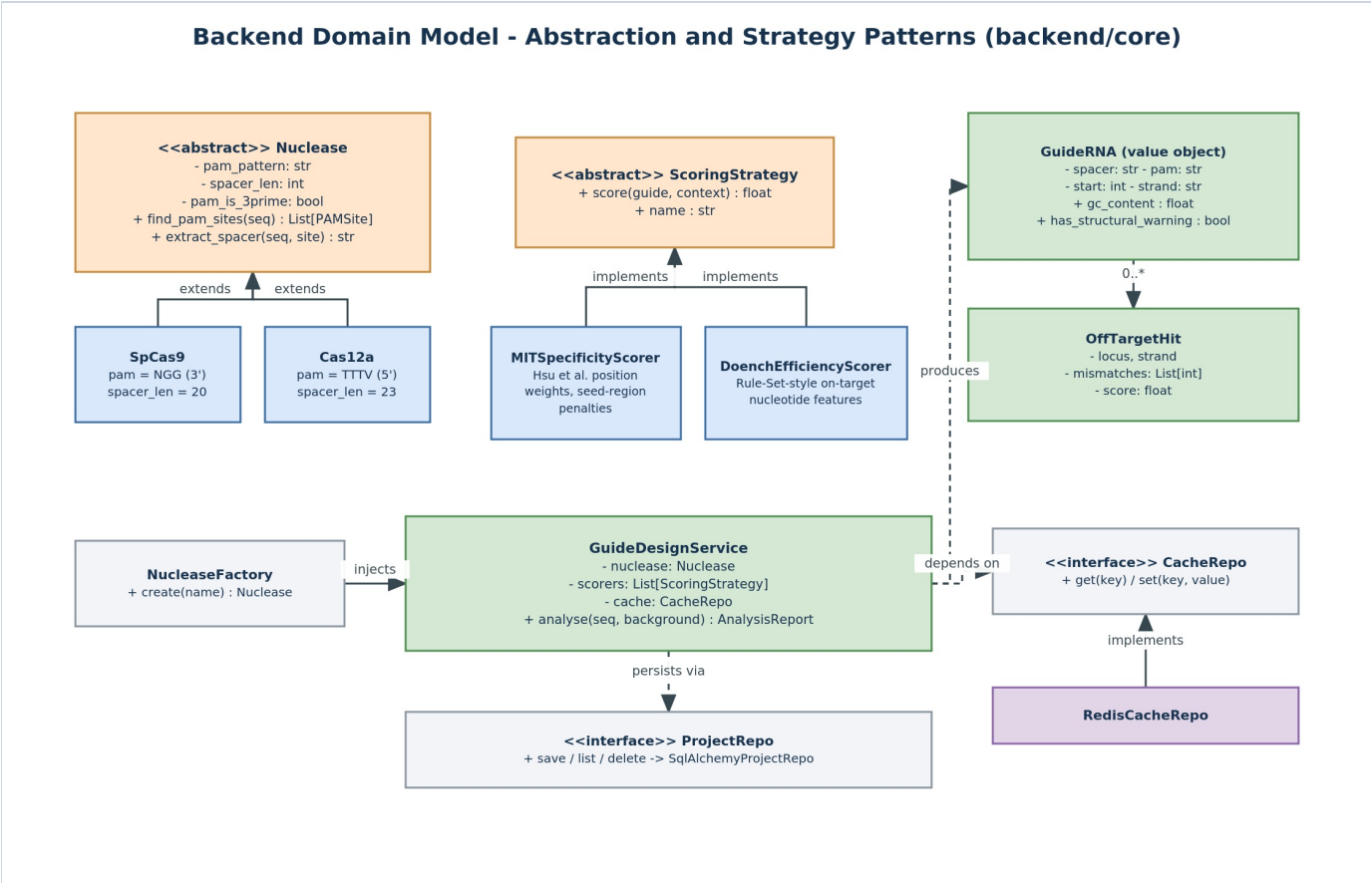


Figure 2 — Domain model: abstract `Nuclease` and `ScoringStrategy` with their concrete implementations, repository interfaces, and the orchestrating service (source: `crispr-architecture.drawio`, page 2).

Runtime View - Run Analysis Request Flow (cache-aside)

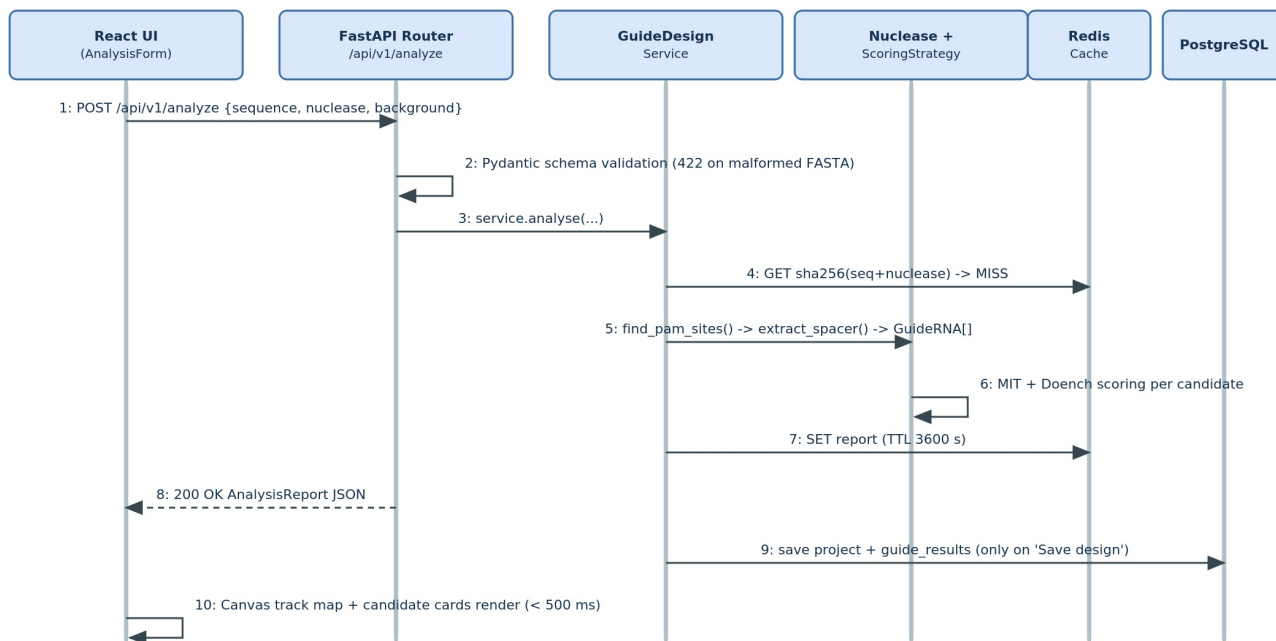
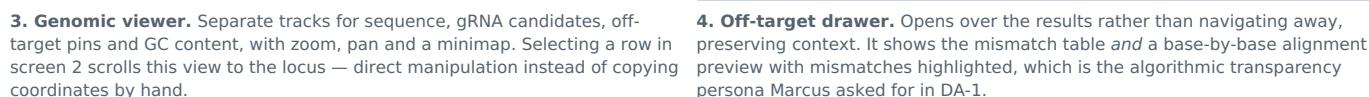
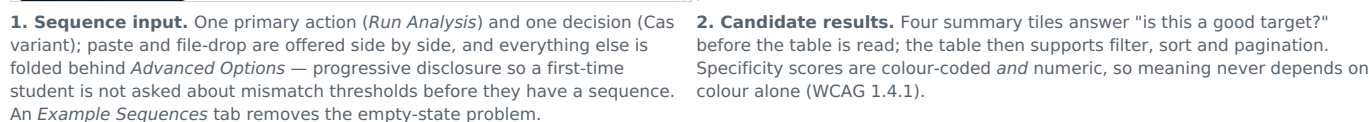
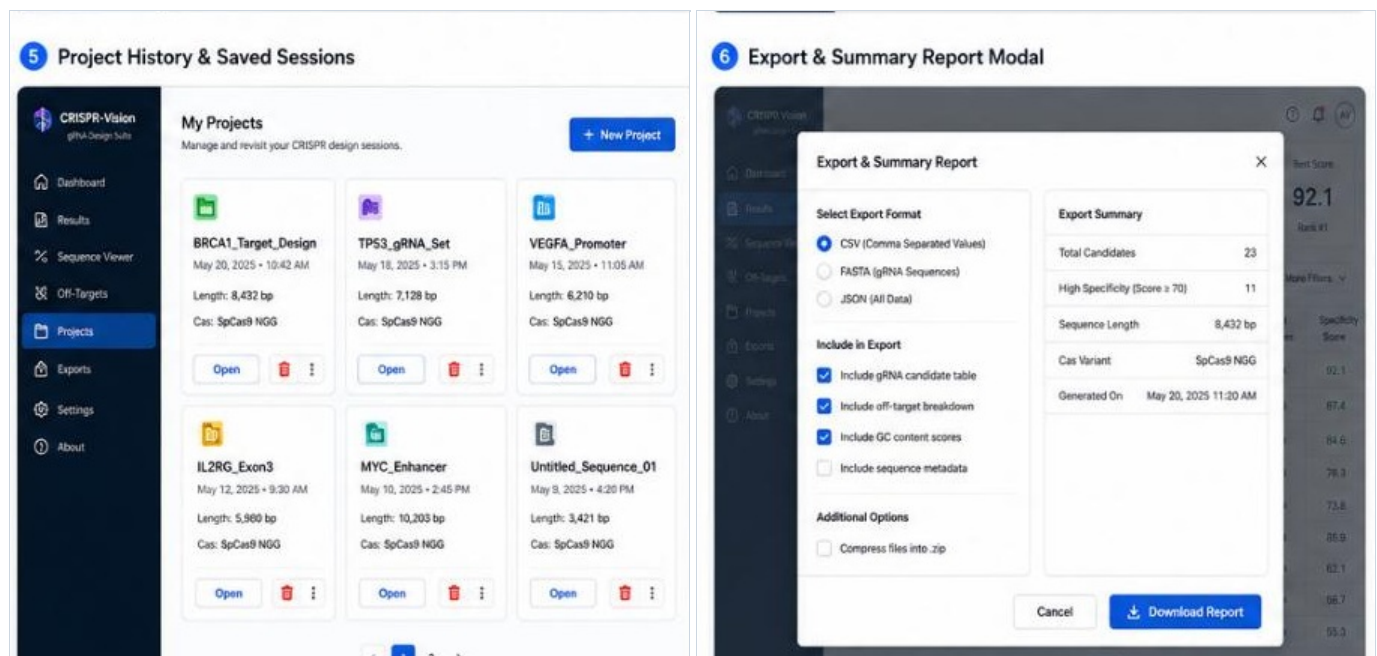


Figure 3 — Runtime view of a "Run Analysis" request, showing validation at the edge, cache-aside lookup, pure-domain computation and optional persistence.

The six screens below are the DA-1 wireframes carried forward with the refinements listed under each one. The layout is a fixed dark navigation rail plus a light data workspace at 1440×900, so the user's spatial memory of "where things are" never resets between steps of the workflow.





5. Project history. Cards carry the metadata a researcher recognises a run by (length, Cas variant, date) rather than an opaque ID — recognition over recall. Destructive delete is separated from *Open* and is confirmed.

6. Export modal. Format choice, content checkboxes and a live summary of exactly what the file will contain, so the user is never surprised by the download. Defaults are pre-selected for the common case.

4.1 What makes it user-friendly

- **One job per screen, one primary button per screen.** The visual hierarchy always makes the next step obvious; secondary actions are text links, not competing buttons.
- **Feedback matched to latency.** Analysis is targeted at under three seconds, so the run button enters a determinate loading state with a progress message instead of freezing; canvas re-render is budgeted at under 500 ms (DA-1 success metrics).
- **Error prevention before error messages.** Non-ACGTN characters are flagged inline as the sequence is pasted, file size and extension are checked before upload, and the server's 422 responses are mapped to plain-language banners (story 25) such as "Line 4 contains an invalid base (X)".
- **Transparency over black boxes.** Every score is accompanied by the evidence behind it — GC content, mismatch positions, alignment preview — so a researcher can defend the choice of guide in a lab meeting.
- **Consistency and reversibility.** Identical colour semantics across table, map and drawer (green = high specificity, amber = caution, red = risk); modals close on Esc; deletes confirm.

5. Design Decisions and Why

Decision	Why	Alternative rejected	Trade-off accepted
1. Abstract Nuclease base class with a factory	PAM rules are the most likely thing to change (Cas12a, Cas12f, PAM-relaxed variants). Isolating them means new enzymes are additive, not invasive.	if variant == "SpCas9" branching inside the endpoint.	One extra indirection to read for the single-nuclease MVP.
2. Strategy pattern for scoring	Specificity (Hsu/MIT) and efficiency (Doench) are independent models that evolve separately and must be comparable; strategies let both run over the same candidate list and let a third be added later.	A single <code>calculate_score()</code> function with flags.	Scores must share one <code>ScoringContext</code> shape, which constrains exotic future models.
3. Cache-aside with Redis keyed by SHA-256 of (sequence + nuclease)	The computation is deterministic and pure, so identical input can always be served from cache; this is what keeps the <3 s metric true when a user re-opens a saved design.	Computing on every request; or caching inside the service as a dict.	A fourth container to run locally, and a one-hour staleness window after an algorithm change (mitigated by versioning the key).
4. Pydantic DTOs as the only frontend-backend contract, published via OpenAPI	Validation happens once, at the edge; the UI never sees a 500 for bad input, and Swagger documents the contract automatically (story 23).	Passing ORM models or raw dicts to the client.	DTOs must be kept in step with domain objects — deliberate duplication in exchange for decoupling.
5. Canvas/SVG track renderer with virtualised drawing, not DOM nodes per base	A 10 kb sequence with thousands of annotations would create tens of thousands of DOM nodes; canvas keeps memory flat and satisfies the DA-1 browser-memory constraint.	One <code></code> per nucleotide, or an off-the-shelf <code>igv.js</code> embed.	Hit-testing and accessibility must be implemented by hand (an accessible data table mirrors the canvas).

6. Maintainability and Future Change

The design was evaluated against the changes that are already on the DA-1 roadmap. In each case the question asked was: *how many modules must be opened?*

Anticipated change	What has to be touched	Why it stays small
Support Cas12a (TTTTV, 5' PAM)	<code>core/nuclease.py</code> + one factory entry + one dropdown option	Behaviour hidden behind the <code>Nuclease</code> contract; callers are unchanged.
Add a newer efficiency model	New <code>ScoringStrategy</code> subclass, registered in the service list	Strategies are independent and additive; ranking already iterates a list.
Whole-genome BLAST/bowtie alignment	<code>core/alignment.py</code> implementation swapped; long runs dispatched to a Celery worker via the same service call	Alignment is already a separate module behind a function signature, not inlined in scoring.
Replace Redis with in-process LRU (or Memcached)	One new <code>CacheRepo</code> implementation; one line in <code>build_service</code>	The domain never imports a cache client.
Add authentication and per-user dashboards	New router + dependency; <code>ProjectRepo</code> gains a <code>user_id</code> filter	Routing and persistence are separate layers; UI reuses the project cards.

Verification strategy. Because `core/` is pure, it is tested with fast unit tests against benchmark FASTA files (the DA-1 "100% match with standard Cas9 regex" metric); services are tested with fake repositories; the API is tested with FastAPI's `TestClient`; and the CI workflow already in the repository runs these plus `docker compose config` validation on every push to `main` and every `feature/*` branch.

6.1 Repository updates accompanying this document

design/	
+ - <code>crispr-architecture.drawio</code>	editable Draw.io source (page 1: architecture, page 2: domain model)
+ - <code>architecture.png</code>	Figure 1 export
+ - <code>class_model.png</code>	Figure 2 export
+ - <code>analysis_flow.png</code>	Figure 3 export
+ - <code>ui/screen1..screen6.png</code>	the six UI screens used in section 4
+ - <code>README-software-design.md</code>	the section appended to the project README

The project `README.md` gains a **Software Design** section that embeds the three diagrams, links the Draw.io source and the UI screens, and summarises the design in three sentences: a layered FastAPI backend with a framework-free domain core, abstractions for nuclease and scoring so new CRISPR systems and scoring models are additive, and a cache-aside Redis layer in front of PostgreSQL to hold the sub-three-second analysis target.